Assignment 6

Collections in Java

Task 1: De-duplication and Sorting
Write a program that:
Accepts a list of integers from the user (including duplicates).
Uses a Set to remove all duplicates.
Prints the unique elements in descending order.

Code Snippet :

FileReading\...\Main.java | collectionsAssognment\...\Main.java | Task 2\...\Main.java | Task3\...\Main.java

```java
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements in the list ");
        int n = sc.nextInt();
        List<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            System.out.print("Enter element " + (i + 1) + ": ");
            int num = sc.nextInt();
            list.add(num);
        }
        System.out.println("\nOriginal list : " + list);
        Set<Integer> set = new HashSet<>(list);
        System.out.println("List removing duplicates " + set);
        List<Integer> uniqueList = new ArrayList<>(set);
        Collections.sort(uniqueList, Collections.reverseOrder());
        System.out.println("List in Descending Order " + uniqueList);
    }
}
```

Output

```
"C:\Program Files\Java\jdk-11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.
Enter the number of elements in the list 8
Enter element 1: 2
Enter element 2: 2
Enter element 3: 2
Enter element 4: 4
Enter element 5: 1
Enter element 6: 4
Enter element 7: 2
Enter element 8: 4

Original list : [2, 2, 2, 4, 1, 4, 2, 4]
List removing duplicates [1, 2, 4]
List in Descending Order [4, 2, 1]

Process finished with exit code 0
```

Task 2: Custom Object Sorting

Create a class Student with the attributes: id, name, and grade.

Initialize a List<Student>.

Implement the Comparable interface to sort students by their grade (highest first).

Use a Comparator to provide an alternative sort by name.

Code Snippet

FileReading\...\Main.java | collectionsAssognment\...\Main.java | Task 2\...\Main.java | rashi.txt

```java
import java.util.*;
class FoodItem implements Comparable<FoodItem> {  10 usages
    int id;  2 usages
    String name;  4 usages
    double price;  4 usages
    FoodItem(int id, String name, double price) {  4 usages
        this.id = id;
        this.name = name;
        this.price = price;
    }
    @Override
    public int compareTo(FoodItem other) {
        return Double.compare(other.price, this.price);
    }
    @Override
    public String toString() {
        return id + "  " + name + "  " + price;
    }
}
```

FileReading\...\Main.java | collectionsAssognment\...\Main.java | Task 2\...\Main.java | rashi.txt

```java
public class Main {
    public static void main(String[] args) {
        items.add(new FoodItem( id: 1,  name: "Burger",  price: 120));
        items.add(new FoodItem( id: 2,  name: "Pizza",  price: 250));
        items.add(new FoodItem( id: 3,  name: "Sandwich",  price: 90));
        items.add(new FoodItem( id: 4,  name: "Pasta",  price: 180));
        Collections.sort(items);
        System.out.println("After Sorting by price");
        for (int i = 0; i < items.size(); i++) {
            System.out.println(items.get(i));
        }
        items.sort(new Comparator<FoodItem>() {
            @Override
            public int compare(FoodItem a, FoodItem b) {
                return a.name.compareTo(b.name);
            }
        });
        System.out.println("\nAfter Sorting By name");
        for (int i = 0; i < items.size(); i++) {
            System.out.println(items.get(i));
        }
    }
}
```

Output

```
"C:\Program Files\Java\jdk-11\bin\java.exe" "-javaagent:C:\Program Files\Je
After Sorting by price
2  Pizza  250.0
4  Pasta  180.0
1  Burger  120.0
3  Sandwich  90.0

After Sorting By name
1  Burger  120.0
4  Pasta  180.0
2  Pizza  250.0
3  Sandwich  90.0
```

Task 3

Linked List Implementation

Provide the pseudo-code for a basic Singly Linked List implementation. Your pseudo-code should clearly outline the following:
The structure of a Node.
The logic for inserting a new element.
The logic for traversing or displaying the list.

Code Snippet

```java
class Node {  6 usages
    int data;  2 usages
    Node next;  5 usages
    Node(int data) {  1 usage
        this.data = data;
        this.next = null;
    }}
class LinkedList {  2 usages
    Node head;  4 usages
    void insert(int value) {  3 usages
        Node newNode = new Node(value);
        if (head == null) {
            head = newNode;
            return;
        }
        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
    void display() {  1 usage
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }}}
public class Main {
    public static void main(String[] args) {
        LinkedList list = new LinkedList();
        list.insert( value: 10);
        list.insert( value: 20);
        list.insert( value: 30);
        list.display();
    }
}
```

Output

```
"C:\Program Files\Java\jdk-11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ
10 20 30
Process finished with exit code 0
```